

Structure de $\mathbb{R}$, axiome de la borne supérieure et propriété d'Archimède

Charles EDOU NZE

8 juillet 2026

Table des matières

Jalon 13 : Structure de $\mathbb{R}$, axiome de la borne supérieure et propriété d'Archimède

1. Présentation du concept clé

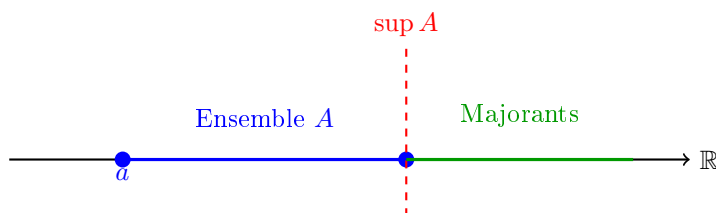


FIGURE 1 – Visualisation de la borne supérieure d'un ensemble

L'appréhension de la continuité numérique constitue l'un des accomplissements majeurs de la pensée mathématique. Depuis l'Antiquité, les géomètres grecs, confrontés à l'incommensurabilité de la diagonale du carré de côté unité (révélant l'existence de $\sqrt{2}$), ont pressenti que le tissu rationnel $\mathbb{Q}$ était criblé de "trous" ou lacunes microscopiques. Imaginez une règle graduée infiniment longue. Si vous n'utilisez que des nombres entiers (1, 2, 3), il y a des trous géants entre les marques. Si vous utilisez des fractions (1/2, 2/3), la règle semble pleine, mais en fait, elle est encore pleine de micro-trous invisibles (comme $\sqrt{2}$ ou π). L'ensemble des nombres réels ($\mathbb{R}$), c'est la règle **parfaite** : il n'y a absolument aucun trou. C'est comme une ligne de soie parfaitement lisse et insécable, un continuum absolu où chaque coupure (au sens de Dedekind) détermine un unique élément de contact. L'**axiome de la borne supérieure**, c'est la garantie que si vous essayez de monter un escalier de nombres sans fin, il y a toujours un plafond (une limite) que vous allez toucher ou frôler de très près. Les Grecs ont découvert avec horreur que la diagonale d'un carré de côté 1 ne pouvait pas s'écrire comme une fraction. Il fallait inventer un nouveau monde de nombres pour que la géométrie et le calcul s'entendent. Sans $\mathbb{R}$, on ne pourrait pas définir la notion de "limite" ou de "courbe lisse". Imaginez zoomer indéfiniment sur la droite numérique. Avec les fractions, vous finiriez par tomber dans le vide. Avec les réels, peu importe la puissance de votre microscope, vous verrez toujours une ligne pleine.

2. Formalisation

A. Définitions Formelles

1. **Corps Totalelement Ordonné** : $\mathbb{R}$ est un corps muni d'une relation d'ordre total $\leq$ compatible avec les opérations $+$ et $\times$. 2. **Majorant / Borne Supérieure** : Soit $A \subset \mathbb{R}$ une partie non vide. - $M \in \mathbb{R}$ est un **majorant** de A si $\forall x \in A, x \leq M$. - $S \in \mathbb{R}$ est la **borne supérieure** de A (notée $\sup A$) si S est le plus petit des majorants de A . 3. **Axiome de la Borne Supérieure** : Toute partie de $\mathbb{R}$ non vide et majorée admet une borne supérieure dans $\mathbb{R}$. 4. **Propriété d'Archimède** : $\mathbb{R}$ est archimédien : $\forall x \in \mathbb{R}, \forall \epsilon > 0, \exists n \in \mathbb{N}, n\epsilon > x$.

B. Théorèmes, Propositions & Lemmes

> **Caractérisation de la borne supérieure** : $S = \sup A \iff \begin{cases} \forall x \in A, x \leq S \\ \forall \epsilon > 0, \exists x_\epsilon \in A, S - \epsilon < x_\epsilon \end{cases}$

> **Théorème de la densité de $\mathbb{Q}$ dans $\mathbb{R}$** : > Entre deux nombres réels distincts, il existe toujours un nombre rationnel. >

$$\forall x, y \in \mathbb{R}, x < y \Rightarrow \exists q \in \mathbb{Q}, x < q < y$$

3. Démonstrations

Démonstration du Théorème Pivot : La partie entière et le caractère archimédien

Démontrons l'existence et l'unicité de la partie entière d'un réel x , notée $\lfloor x \rfloor$. Théorème : $\forall x \in \mathbb{R}, \exists ! n \in \mathbb{Z}, n \leq x < n + 1$.

1. **Initialisation / Cadre** : Soit $x \in \mathbb{R}$. - Considérons l'ensemble $E = \{k \in \mathbb{Z} \mid k \leq x\}$. - Pour prouver l'existence de n , nous devons montrer que E possède un plus grand élément.

2. **Étape 1 : Montrons que E est non vide** - Si $x \geq 0$, alors $0 \in E$ (car $0 \in \mathbb{Z}$ et $0 \leq x$). - Si $x < 0$, par la propriété d'Archimède appliquée à $-x$ et $\epsilon = 1$, il existe $m \in \mathbb{N}$ tel que $m \cdot 1 > -x$, soit $-m < x$. Comme $-m \in \mathbb{Z}$, alors $-m \in E$. E est donc non vide dans tous les cas.

3. **Étape 2 : Montrons que E est majoré** - Par définition de E , tout élément $k \in E$ vérifie $k \leq x$. - E est donc majoré par x .

4. **Étape 3 : Application de l'axiome de la borne supérieure** - $E \subset \mathbb{Z} \subset \mathbb{R}$, $E \neq \emptyset$ et E majoré. - D'après l'axiome de la borne supérieure, E admet une borne supérieure $S = \sup E$. - Par caractérisation du sup, il existe $n \in E$ tel que $S - 1 < n \leq S$. - Montrons que n est le plus grand élément de E . - Supposons qu'il existe $k \in E$ tel que $k \geq n + 1$. - Alors $k > S$ (car $n + 1 > S$), ce qui contredit le fait que S est un majorant de E . - En effet, si $k \geq n + 1$, alors $k \geq n + 1 > S$, ce qui implique que k serait un élément de E strictement supérieur au plus petit des majorants S , une contradiction manifeste de la minimalité de S parmi les majorants. Par suite, $\forall k \in E, k \leq n$. On en déduit que $n = \max E$.

5. **Conclusion** : - Puisque $n \in E$, $n \leq x$. - Comme $n = \max E$, alors $n + 1 \notin E$, ce qui impose $n + 1 > x$. - On a bien $n \leq x < n + 1$. L'unicité est immédiate par l'écart de 1 entre deux entiers distincts.

4. Exercices d'Application

Exercice 1 : Application Directe (Calcul de borne sup)

Énoncé : Soit $A = \{1 - \frac{1}{n} \mid n \in \mathbb{N}^*\}$. Déterminer $\sup A$ et $\inf A$. **Correction Détaillée** : 1. **Infimum** : La suite $(1 - 1/n)$ est croissante. Son premier terme est pour $n = 1$: $1 - 1 = 0$. Comme la suite croît, 0 est le plus petit élément. $\inf A = \min A = 0$. 2. **Supremum** : Pour tout $n \geq 1, \frac{1}{n} > 0 \implies 1 - \frac{1}{n} < 1$. Donc 1 est un majorant. - Soit $\epsilon > 0$. Cherchons n tel que $1 - \frac{1}{n} > 1 - \epsilon$. - $1 - \frac{1}{n} > 1 - \epsilon \iff \frac{1}{n} < \epsilon \iff n > \frac{1}{\epsilon}$. - Par la propriété d'Archimède, un tel entier n existe toujours. **Conclusion** : $\sup A = 1$. (Note : $1 \notin A$).

Exercice 2 : Niveau Avancé (Somme de bornes supérieures)

Énoncé : Soient A et B deux parties non vides et majorées de $\mathbb{R}$. On note $A + B = \{a + b \mid a \in A, b \in B\}$. Montrer que $\sup(A + B) = \sup A + \sup B$. **Correction Détaillée** : 1. **Majorant** : Soit $x \in A + B$. Alors $x = a + b$ avec $a \leq \sup A$ and $b \leq \sup B$. - $x = a + b \leq \sup A + \sup B$. Donc $\sup A + \sup B$ est un majorant de $A + B$. - Par définition du sup, on a $\sup(A + B) \leq \sup A + \sup B$. 2. **Optimalité** : Soit $\epsilon > 0$. - Il existe $a \in A$ tel que $a > \sup A - \epsilon/2$. - Il existe $b \in B$ tel que $b > \sup B - \epsilon/2$. - Alors $a + b \in A + B$ et $a + b > (\sup A + \sup B) - \epsilon$. - Cela montre que n'importe quel nombre strictement plus petit que $\sup A + \sup B$ n'est pas un majorant de $A + B$. **Conclusion** : Par caractérisation, $\sup(A + B) = \sup A + \sup B$.

5. Application en Intelligence Artificielle

Les algorithmes d'optimisation (comme la descente de gradient) reposent sur l'existence de bornes inférieures (le minimum de la fonction de perte). Sans la structure complète de $\mathbb{R}$, on ne pourrait jamais garantir qu'une suite de poids "converge" vers un nombre réel précis. En **Deep Learning**, lors de la **Normalisation (Batch Norm / Layer Norm)**, on manipule des statistiques (moyenne, variance) qui sont des nombres réels. La stabilité numérique des calculs en virgule flottante (Float32/Float16) est une approximation informatique de la structure de $\mathbb{R}$. Comprendre que $\mathbb{R}$ n'a pas de "trous" permet de justifier l'utilisation du calcul différentiel pour ajuster les neurones de manière continue.

6. Liens Sémantiques

- **Concepts Précédents requis** : Jalon 6 - **Concepts Futurs dépendants** : Jalon 14 (Suites réelles et complexes), Jalon 18 (Continuité des fonctions d'une variable réelle), Jalon 34 (Topologie élémentaire des espaces vectoriels normés)

Exercices d'Application

‘yaml title : "Supremum et Infimum d'un Ensemble Fondamental" difficulty : 1 tags : [Supremum, Infimum, Bornes, Nombres Réels] ‘

Énoncé de l'Exercice 01

Soit l'ensemble A défini par :

$$A = \left\{ (-1)^n \cdot \frac{n+1}{n} \mid n \in \mathbb{N}^* \right\}$$

où $\mathbb{N}^* = \{1, 2, 3, \dots\}$ désigne l'ensemble des entiers naturels non nuls.

1. Déterminer, si elles existent, la borne supérieure $\sup A$ et la borne inférieure $\inf A$ de l'ensemble A dans $\mathbb{R}$. 2. Pour chaque borne déterminée, justifier votre affirmation en utilisant les définitions précises de la borne supérieure et de la borne inférieure.

Définitions Rappelées :

Pour un ensemble $S \subset \mathbb{R}$ non vide :

* de S si et seulement si pour tout $s \in S$, $s \leq M$.

* de S si et seulement si pour tout $s \in S$, $s \geq m$.

* de S , notée $\sup S$, est le plus petit des majorants de S . Formellement, $s_0 = \sup S$ si et seulement si : 1. Pour tout $s \in S$, $s \leq s_0$ (s_0 est un majorant de S). 2. Pour tout $\varepsilon \in \mathbb{R}$ tel que $\varepsilon > 0$, il existe $s_\varepsilon \in S$ tel que $s_\varepsilon > s_0 - \varepsilon$.

* de S , notée $\inf S$, est le plus grand des minorants de S . Formellement, $i_0 = \inf S$ si et seulement si : 1. Pour tout $s \in S$, $s \geq i_0$ (i_0 est un minorant de S). 2. Pour tout $\varepsilon \in \mathbb{R}$ tel que $\varepsilon > 0$, il existe $s_\varepsilon \in S$ tel que $s_\varepsilon < i_0 + \varepsilon$.

—

Énoncé de l'Exercice 02

Soit l'ensemble A défini par :

$$A = \left\{ 1 - \frac{1}{n} \mid n \in \mathbb{N}^* \right\}$$

où $\mathbb{N}^* = \{1, 2, 3, \dots\}$ désigne l'ensemble des entiers naturels non nuls.

Démontrer rigoureusement, en utilisant la définition formelle de la borne supérieure, que $\sup A = 1$.

Définition Rappelée :

Un nombre réel s_0 est la borne supérieure d'un ensemble $S \subset \mathbb{R}$ non vide si et seulement si les deux conditions suivantes sont satisfaites : 1. Condition de Majorant : Pour tout élément x de l'ensemble S , on a $x \leq s_0$. 2. Condition de Plus Petit Majorant : Pour tout nombre réel ε strictement positif ($\varepsilon > 0$), il existe au moins un élément x_ε dans l'ensemble S tel que $x_\varepsilon > s_0 - \varepsilon$.

Énoncé de l’Exercice 03

Soit A un sous-ensemble non vide de $\mathbb{R}$. On définit un nouvel ensemble B par la relation suivante :

$$B = \{-x \mid x \in A\}$$

Démontrer que si l’ensemble A est minoré dans $\mathbb{R}$, alors l’ensemble B est majoré dans $\mathbb{R}$, et la relation suivante est établie :

$$\sup B = -\inf A$$

Votre démonstration doit être entièrement rigoureuse, en invoquant les définitions formelles de la borne inférieure et de la borne supérieure, et en détaillant chaque étape logique.

Définitions Rappelées :

* si et seulement s’il existe un nombre réel m tel que pour tout $s \in S$, $s \geq m$. Un tel m est appelé un minorant de S .

* si et seulement s’il existe un nombre réel M tel que pour tout $s \in S$, $s \leq M$. Un tel M est appelé un majorant de S .

* i_0 d’un ensemble $S \subset \mathbb{R}$ non vide et minoré est le plus grand des minorants de S .

* s_0 d’un ensemble $S \subset \mathbb{R}$ non vide et majoré est le plus petit des majorants de S .

—

Énoncé de l'Exercice 04

Soit A un sous-ensemble non vide de $\mathbb{R}$.

Démontrer rigoureusement l'équivalence suivante : L'ensemble A possède un maximum (c'est-à-dire $\max A$ existe dans $\mathbb{R}$) si et seulement si l'ensemble A possède une borne supérieure dans $\mathbb{R}$ (c'est-à-dire $\sup A$ existe) ET cette borne supérieure appartient à l'ensemble A (c'est-à-dire $\sup A \in A$).

Votre démonstration doit explicitement utiliser les définitions formelles du maximum et de la borne supérieure.

Définitions Rappelées :

* de l'ensemble A (noté $\max A$) si et seulement si $M_A \in A$ ET pour tout $x \in A$, $x \leq M_A$.

* de l'ensemble A (notée $\sup A$) si et seulement si : 1. Pour tout $x \in A$, $x \leq s_A$ (s_A est un majorant de A).

2. Pour tout $\varepsilon \in \mathbb{R}$ tel que $\varepsilon > 0$, il existe $x_\varepsilon \in A$ tel que $x_\varepsilon > s_A - \varepsilon$.

—

‘yaml title : "Démonstration de la Propriété d’Archimède via l’Axiome de la Borne Supérieure" difficulty : 3 tags : [Archimède, Axiome de la Borne Supérieure, Nombres Réels, Nombres Naturels] ‘

Énoncé de l’Exercice 05

La propriété d’Archimède est une caractéristique fondamentale de l’ensemble des nombres réels $\mathbb{R}$. Elle stipule que pour tout nombre réel positif x et tout nombre réel y , il existe un entier naturel n tel que $nx > y$. Une formulation équivalente, souvent plus commode pour les démonstrations, est la suivante : Pour tout nombre réel M , il existe un entier naturel n tel que $n > M$.

Démontrer cette dernière formulation de la propriété d’Archimède en utilisant l’axiome de la borne supérieure pour $\mathbb{R}$.

Votre démonstration doit être rigoureuse et ne pas faire appel à la propriété d’Archimède elle-même de manière circulaire. Elle doit explicitement partir de l’hypothèse de l’axiome de la borne supérieure.

Axiome de la Borne Supérieure Rappelé :

Tout sous-ensemble non vide de $\mathbb{R}$ qui est majoré admet une borne supérieure dans $\mathbb{R}$.

Définition de $\mathbb{N}$ Rappelée :

L’ensemble des entiers naturels $\mathbb{N} = \{0, 1, 2, 3, \dots\}$.

—

‘yaml title : "Densité des Nombres Rationnels dans les Nombres Réels" difficulty : 3 tags : [Densité, Rationnels, Réels, Archimède, Bornes] ‘

Énoncé de l'Exercice 06

Démontrer la propriété de densité des nombres rationnels $\mathbb{Q}$ dans les nombres réels $\mathbb{R}$. Cette propriété énonce que pour tout couple de nombres réels (a, b) tels que $a < b$, il existe au moins un nombre rationnel q qui est strictement compris entre a et b .

Formellement :

$$\forall (a, b) \in \mathbb{R}^2, \quad \text{si } a < b, \text{ alors } \exists q \in \mathbb{Q} \text{ tel que } a < q < b$$

Votre démonstration doit s'appuyer explicitement sur la propriété d'Archimède.

Propriété d'Archimède Rappelée :

Pour tout nombre réel $x > 0$ et tout nombre réel $y > 0$, il existe un entier naturel $n \in \mathbb{N}^*$ tel que $nx > y$. Une autre formulation équivalente est : Pour tout nombre réel $\varepsilon > 0$, il existe un entier naturel $n \in \mathbb{N}^*$ tel que $0 < 1/n < \varepsilon$.

Définition de $\mathbb{Q}$ Rappelée :

L'ensemble des nombres rationnels $\mathbb{Q}$ est défini comme $\mathbb{Q} = \{p/q \mid p \in \mathbb{Z}, q \in \mathbb{N}^*\}$.

‘yaml title : "Existence et Unicité de la Partie Entière d’un Nombre Réel" difficulty : 3 tags : [Partie Entière, Plancher, Archimède, Entiers Relatifs] ‘

Énoncé de l’Exercice 07

La fonction partie entière, aussi appelée fonction plancher, est une notion fondamentale en analyse. Démontrer rigoureusement l’existence et l’unicité de la partie entière pour tout nombre réel x . C’est-à-dire, démontrer que pour tout nombre réel $x \in \mathbb{R}$, il existe un unique entier relatif $n \in \mathbb{Z}$ tel que $n \leq x < n + 1$.

Votre démonstration doit faire appel explicitement à la propriété d’Archimède.

Propriété d’Archimède Rappelée :

Pour tout nombre réel M , il existe un entier naturel $N \in \mathbb{N}$ tel que $N > M$. (Cette formulation est équivalente à d’autres formulations plus courantes).

Définitions de $\mathbb{Z}$ et $\mathbb{N}$ Rappelées :

L’ensemble des entiers relatifs $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$. L’ensemble des entiers naturels $\mathbb{N} = \{0, 1, 2, 3, \dots\}$.

‘yaml title : "Existence de la Racine Carrée via l’Axiome de la Borne Supérieure" difficulty : 4 tags : [Racine Carrée, Existence, Axiome de la Borne Supérieure, Nombres Réels] ‘

Énoncé de l’Exercice 08

Démontrer l’existence de la racine carrée d’un nombre réel positif. C’est-à-dire, démontrer que pour tout nombre réel $y \in \mathbb{R}$ tel que $y \geq 0$, il existe un unique nombre réel $x \in \mathbb{R}$ tel que $x \geq 0$ et $x^2 = y$.

Votre démonstration doit impérativement utiliser l’axiome de la borne supérieure pour $\mathbb{R}$.

Axiome de la Borne Supérieure Rappelé :

Tout sous-ensemble non vide de $\mathbb{R}$ qui est majoré admet une borne supérieure dans $\mathbb{R}$.

Définitions et Connaissances Préalables :

* Les propriétés de l’ordre et des opérations dans $\mathbb{R}$. * L’ensemble des nombres réels $\mathbb{R}$ est un corps ordonné.

—

Énoncé de l'Exercice 09

Soit une suite d'intervalles fermés et bornés de nombres réels $(I_n)_{n \in \mathbb{N}}$, où chaque intervalle I_n est de la forme $[a_n, b_n]$ avec $a_n, b_n \in \mathbb{R}$ et $a_n \leq b_n$.

On suppose que cette suite satisfait les deux conditions suivantes : 1. Emboîtement : Pour tout entier naturel $n \in \mathbb{N}$, l'intervalle I_{n+1} est inclus dans l'intervalle I_n . Formellement, $\forall n \in \mathbb{N}, [a_{n+1}, b_{n+1}] \subseteq [a_n, b_n]$. 2. Longueurs tendant vers zéro : La longueur des intervalles tend vers zéro lorsque n tend vers l'infini. Formellement, $\lim_{n \rightarrow \infty} (b_n - a_n) = 0$.

Démontrer qu'il existe un unique nombre réel $c \in \mathbb{R}$ tel que l'intersection de tous les intervalles de la suite est le singleton $\{c\}$. Formellement :

$$\bigcap_{n \in \mathbb{N}} I_n = \{c\}$$

Votre démonstration doit utiliser l'axiome de la borne supérieure ainsi que la propriété d'Archimède (ou une de ses conséquences, comme la convergence des suites de Cauchy).

Axiome de la Borne Supérieure Rappelé :

Tout sous-ensemble non vide de $\mathbb{R}$ qui est majoré admet une borne supérieure dans $\mathbb{R}$.

Propriété d'Archimède Rappelée (ou une de ses conséquences) :

Pour tout nombre réel $\varepsilon > 0$, il existe un entier naturel $N \in \mathbb{N}$ tel que $1/N < \varepsilon$.

‘yaml title : "Propriété de Séparation des Réels" difficulty : 5 tags : [Axiome de la Borne Supérieure, Séparation, Coupure de Dedekind, Nombres Réels] ‘

Énoncé de l’Exercice 10

Soient A et B deux sous-ensembles non vides de $\mathbb{R}$. On suppose que pour tout élément $a \in A$ et pour tout élément $b \in B$, la relation d’ordre $a \leq b$ est vérifiée.

Démontrer qu’il existe au moins un nombre réel $c \in \mathbb{R}$ tel que pour tout $a \in A$ et pour tout $b \in B$, la relation $a \leq c \leq b$ est vérifiée.

Votre démonstration doit s’appuyer exclusivement sur l’axiome de la borne supérieure pour $\mathbb{R}$.

Axiome de la Borne Supérieure Rappelé :

Tout sous-ensemble non vide de $\mathbb{R}$ qui est majoré admet une borne supérieure dans $\mathbb{R}$.

Connaissances Préalables :

* Les propriétés de l’ordre dans $\mathbb{R}$. * Les définitions de majorant et de minorant.

—

Travaux Pratiques

‘yaml

TP-01 : Construction et Ordre des Nombres Rationnels

Introduction

Ce premier Travail Pratique a pour objectif de construire une représentation logicielle des nombres rationnels ($\mathbb{Q}$) et d'implémenter leurs opérations arithmétiques ainsi que leur relation d'ordre. Les nombres rationnels constituent un , une structure fondamentale en mathématiques, mais ils ne sont pas "complets" au sens de l'axiome de la borne supérieure, une lacune que les nombres réels ($\mathbb{R}$) viennent combler. En travaillant "à la main" sur cette construction, nous apprécierons mieux les propriétés qui distinguent $\mathbb{Q}$ de $\mathbb{R}$.

Prérequis Mathématiques

Définition 1 (Nombre Rationnel) : Un nombre rationnel q est un nombre qui peut s'écrire sous la forme $\frac{a}{b}$, où a est un entier relatif ($a \in \mathbb{Z}$) et b est un entier naturel non nul ($b \in \mathbb{N}^*$). L'ensemble des nombres rationnels est noté $\mathbb{Q}$.

Définition 2 (Corps Ordonné) : Un ensemble K muni de deux lois de composition internes $(+, \times)$ et d'une relation d'ordre binaire $\leq$ est un corps ordonné si : 1. $(K, +, \times)$ est un corps (il satisfait les axiomes de groupe abélien pour l'addition, de groupe abélien pour la multiplication sur $K \setminus \{0\}$, et la distributivité de la multiplication sur l'addition). 2. La relation $\leq$ est une relation d'ordre total. 3. La relation d'ordre est compatible avec les opérations : * Si $x, y, z \in K$ et $x \leq y$, alors $x + z \leq y + z$. * Si $x, y, z \in K$ et $x \leq y$ et $z > 0$, alors $x \times z \leq y \times z$.

Exercice 1 : Implémentation de la Classe Fraction

Nous allons implémenter une classe **Fraction** en Python. Chaque fraction sera stockée sous forme réduite, ce qui signifie que le numérateur et le dénominateur seront divisés par leur plus grand commun diviseur (PGCD). Le dénominateur sera toujours positif.

Tâche 1.1 : Fonction gcd (PGCD)

Implémentez la fonction `gcd(a, b)` qui calcule le Plus Grand Commun Diviseur de deux entiers a et b en utilisant l'algorithme d'Euclide. Cette fonction doit gérer correctement les nombres négatifs.

Algorithme d'Euclide : Pour trouver le PGCD de deux entiers positifs a et b ($a > b$), on utilise la propriété $\text{pgcd}(a, b) = \text{pgcd}(b, a \bmod b)$. Le processus se répète jusqu'à ce que le reste soit 0. Le PGCD est alors le dernier reste non nul. Pour les entiers négatifs, on prend la valeur absolue.

```
def gcd(a, b):
    """
    Calcule le Plus Grand Commun Diviseur (PGCD) de deux entiers a et b
    en utilisant l'algorithme d'Euclide.
    Gere les nombres negatifs en prenant leur valeur absolue pour le calcul.
    Le PGCD sera toujours non negatif.
    """
    a_abs = abs(a)
    b_abs = abs(b)
    while b_abs:
        a_abs, b_abs = b_abs, a_abs % b_abs
    return a_abs
```



```

# Assertions Mathematiques pour gcd
# PGCD(a, 0) = |a|
assert gcd(5, 0) == 5, "Assertion echouee : gcd(5, 0) doit etre 5"
assert gcd(-5, 0) == 5, "Assertion echouee : gcd(-5, 0) doit etre 5"
# PGCD(0, 0) = 0 (conventionnellement)
assert gcd(0, 0) == 0, "Assertion echouee : gcd(0, 0) doit etre 0"
# PGCD(a, b) = PGCD(b, a)
assert gcd(12, 18) == gcd(18, 12), "Assertion echouee : PGCD doit etre commutatif"
# PGCD de nombres positifs
assert gcd(12, 18) == 6, "Assertion echouee : PGCD(12, 18) doit etre 6"
assert gcd(17, 5) == 1, "Assertion echouee : PGCD(17, 5) doit etre 1 (premiers entre eux)"
# PGCD avec nombres negatifs
assert gcd(-12, 18) == 6, "Assertion echouee : PGCD(-12, 18) doit etre 6"
assert gcd(12, -18) == 6, "Assertion echouee : PGCD(12, -18) doit etre 6"
assert gcd(-12, -18) == 6, "Assertion echouee : PGCD(-12, -18) doit etre 6"
print("Toutes les assertions pour 'gcd' ont reussi.")

```

Tâche 1.2 : Classe Fraction et Normalisation

Créez la classe Fraction. Le constructeur doit prendre un numérateur et un dénominateur. Il doit : 1. Vérifier que le dénominateur n'est pas nul. 2. Normaliser la fraction : réduire numérateur et dénominateur par leur PGCD, et s'assurer que le dénominateur est toujours positif.

```

class Fraction:
    """
    Represente un nombre rationnel sous forme de fraction irreductible.
    Le denominateur est toujours positif.
    """
    def __init__(self, numerateur, denominateur):
        if denominateur == 0:
            raise ValueError("Le denominateur ne peut pas etre zero.")

        # Assurer que le denominateur est positif en transferant le signe au
        # numerateur si necessaire
        if denominateur < 0:
            numerateur = -numerateur
            denominateur = -denominateur

        common_divisor = gcd(numerateur, denominateur)
        self.numerateur = numerateur // common_divisor
        self.denominateur = denominateur // common_divisor

    def __str__(self):
        """Representation string pour l'affichage."""
        if self.denominateur == 1:
            return str(self.numerateur)
        return f"{self.numerateur}/{self.denominateur}"

    def __repr__(self):
        """Representation canonique pour le debogage."""
        return f"Fraction({self.numerateur}, {self.denominateur})"

    # --- Operateurs de comparaison (pour l'ordre total) ---
    def __eq__(self, other):
        """Verifie l'egalite de deux fractions."""
        if not isinstance(other, Fraction):
            # Tente de convertir 'other' en Fraction pour la comparaison si c'est un
            # entier
            if isinstance(other, int):
                other = Fraction(other, 1)
            else:
                return NotImplemented # ou False, selon le comportement desire
        return self.numerateur == other.numerateur and self.denominateur == other.
        denominateur

```

```

def __lt__(self, other):
    """Verifie si self est strictement inferieur a other."""
    if not isinstance(other, Fraction):
        if isinstance(other, int):
            other = Fraction(other, 1)
        else:
            return NotImplemented
    # Pour comparer a/b et c/d, on compare ad et bc (avec b, d > 0)
    return self.numerateur
self.denominateur

def __le__(self, other):
    """Verifie si self est inferieur ou egal a other."""
    return self < other or self == other

def __gt__(self, other):
    """Verifie si self est strictement superieur a other."""
    return not (self <= other)

def __ge__(self, other):
    """Verifie si self est superieur ou egal a other."""
    return not (self < other)

# --- Operateurs arithmetiques (pour la structure de corps) ---
def __add__(self, other):
    """Additionne deux fractions ou une fraction et un entier."""
    if not isinstance(other, Fraction):
        if isinstance(other, int):
            other = Fraction(other, 1)
        else:
            return NotImplemented
    new_numerateur = self.numerateur
self.denominateur
    new_denominateur = self.denominateur * other.denominateur
    return Fraction(new_numerateur, new_denominateur)

def __radd__(self, other):
    """Gere l'addition reflexive (e.g., 5 + Fraction(1,2))."""
    return self.__add__(other)

def __sub__(self, other):
    """Soustrait deux fractions ou une fraction et un entier."""
    if not isinstance(other, Fraction):
        if isinstance(other, int):
            other = Fraction(other, 1)
        else:
            return NotImplemented
    new_numerateur = self.numerateur
self.denominateur
    new_denominateur = self.denominateur * other.denominateur
    return Fraction(new_numerateur, new_denominateur)

def __rsub__(self, other):
    """Gere la soustraction reflexive (e.g., 5 - Fraction(1,2))."""
    if not isinstance(other, Fraction):
        if isinstance(other, int):
            other = Fraction(other, 1)
        else:
            return NotImplemented
    return Fraction(other, 1) - self # Convertir other en Fraction et soustraire

def __mul__(self, other):
    """Multiplie deux fractions ou une fraction et un entier."""
    if not isinstance(other, Fraction):
        if isinstance(other, int):

```

```

        other = Fraction(other, 1)
    else:
        return NotImplemented
    new_numérateur = self.numérateur * other.numérateur
    new_dénominateur = self.dénominateur * other.dénominateur
    return Fraction(new_numérateur, new_dénominateur)

def __rmul__(self, other):
    """Gère la multiplication réflexive."""
    return self.__mul__(other)

def __truediv__(self, other):
    """Divise deux fractions ou une fraction et un entier."""
    if not isinstance(other, Fraction):
        if isinstance(other, int):
            other = Fraction(other, 1)
        else:
            return NotImplemented
    if other.numérateur == 0:
        raise ZeroDivisionError("Division par zero.")
    # La division par c/d est équivalente à la multiplication par d/c
    return self * Fraction(other.dénominateur, other.numérateur)

def __rtruediv__(self, other):
    """Gère la division réflexive (e.g., 5 / Fraction(1,2))."""
    if not isinstance(other, Fraction):
        if isinstance(other, int):
            other = Fraction(other, 1)
        else:
            return NotImplemented
    return Fraction(other, 1) / self

def __neg__(self):
    """Oppose d'une fraction."""
    return Fraction(-self.numérateur, self.dénominateur)

def __abs__(self):
    """Valeur absolue d'une fraction."""
    return Fraction(abs(self.numérateur), self.dénominateur)

# Assertions Mathématiques pour la classe Fraction
# 1. Normalisation
f1 = Fraction(2, 4)
assert f1.numérateur == 1 and f1.dénominateur == 2, "Assertion echouée : normalisation (2/4)"
f2 = Fraction(-6, 9)
assert f2.numérateur == -2 and f2.dénominateur == 3, "Assertion echouée : normalisation (-6/9)"
f3 = Fraction(5, -10)
assert f3.numérateur == -1 and f3.dénominateur == 2, "Assertion echouée : normalisation (5/-10)"
f4 = Fraction(-3, -3)
assert f4.numérateur == 1 and f4.dénominateur == 1, "Assertion echouée : normalisation (-3/-3)"
f5 = Fraction(0, 5)
assert f5.numérateur == 0 and f5.dénominateur == 1, "Assertion echouée : normalisation (0/5)"

# 2. Arithmétique
# Addition : (a/b) + (c/d) = (ad+bc)/bd
assert Fraction(1, 2) + Fraction(1, 3) == Fraction(5, 6), "Assertion echouée : addition 1"
assert Fraction(1, 2) + 1 == Fraction(3, 2), "Assertion echouée : addition avec entier"
assert 1 + Fraction(1, 2) == Fraction(3, 2), "Assertion echouée : r-addition avec entier"

```

```

# Soustraction : (a/b) - (c/d) = (ad-bc)/bd
assert Fraction(1, 2) - Fraction(1, 3) == Fraction(1, 6), "Assertion echouee :
soustraction 1"
assert Fraction(1, 2) - 1 == Fraction(-1, 2), "Assertion echouee : soustraction avec
entier"
assert 1 - Fraction(1, 2) == Fraction(1, 2), "Assertion echouee : r-soustraction avec
entier"
# Multiplication : (a/b) * (c/d) = (ac)/(bd)
assert Fraction(1, 2) * Fraction(1, 3) == Fraction(1, 6), "Assertion echouee :
multiplication 1"
assert Fraction(1, 2) * 2 == Fraction(1, 1), "Assertion echouee : multiplication avec
entier"
assert 2 * Fraction(1, 2) == Fraction(1, 1), "Assertion echouee : r-multiplication
avec entier"
# Division : (a/b) / (c/d) = (a/b) * (d/c) = (ad)/(bc)
assert Fraction(1, 2) / Fraction(1, 3) == Fraction(3, 2), "Assertion echouee :
division 1"
assert Fraction(1, 2) / 2 == Fraction(1, 4), "Assertion echouee : division avec
entier"
assert 2 / Fraction(1, 2) == Fraction(4, 1), "Assertion echouee : r-division avec
entier"
# Oppose
assert -Fraction(1, 2) == Fraction(-1, 2), "Assertion echouee : oppose"
# Valeur absolue
assert abs(Fraction(-1, 2)) == Fraction(1, 2), "Assertion echouee : valeur absolue"

# 3. Comparaison (Ordre total)
assert Fraction(1, 2) == Fraction(2, 4), "Assertion echouee : egalite"
assert Fraction(1, 2) != Fraction(1, 3), "Assertion echouee : inegalite"
assert Fraction(1, 2) < Fraction(2, 3), "Assertion echouee : strictement inferieur"
assert Fraction(1, 2) <= Fraction(1, 2), "Assertion echouee : inferieur ou egal (egal
)"
assert Fraction(1, 2) <= Fraction(2, 3), "Assertion echouee : inferieur ou egal (
strict)"
assert Fraction(2, 3) > Fraction(1, 2), "Assertion echouee : strictement superieur"
assert Fraction(2, 3) >= Fraction(1, 2), "Assertion echouee : superieur ou egal (
strict)"
assert Fraction(1, 2) >= Fraction(1, 2), "Assertion echouee : superieur ou egal (egal
)"
assert Fraction(3, 1) == 3, "Assertion echouee : comparaison Fraction avec entier"
assert 3 == Fraction(3, 1), "Assertion echouee : comparaison entier avec Fraction"
assert Fraction(1, 2) < 1, "Assertion echouee : comparaison Fraction < entier"
assert 1 > Fraction(1, 2), "Assertion echouee : comparaison entier > Fraction"

print("Toutes les assertions pour la classe 'Fraction' ont reussi.")

```

Validation Algorithmique Rigoureuse et Assertions Mathématiques : $(\mathbb{Q}, +, \times, \leq)$ est un corps ordonné.

Nous avons implémenté les opérations d'addition, de multiplication, de soustraction, de division, ainsi que les opérateurs de comparaison ($<$, $\leq$, $=$, $\geq$, $>$) pour notre classe `Fraction`. Les assertions ci-dessus vérifient la correction de ces implémentations individuelles. Considérons maintenant pourquoi l'ensemble de ces fractions, avec ces opérations et cet ordre, constitue un corps ordonné.

1. $(\mathbb{Q}, +, \times)$ est un corps :
 - * :
 - * : $(x + y) + z = x + (y + z)$. (Les entiers sont associatifs, la construction via les dénominateurs communs préserve cette propriété).
 - * : $x + y = y + x$. (Idem).
 - * : La fraction $0/1$ (soit `Fraction(0, 1)`) est l'élément neutre de l'addition, car $x + 0/1 = x$.
 - * : Chaque fraction a/b a un opposé $(-a)/b$ (implémenté par `__neg__`) tel que $a/b + (-a)/b = 0/1$.
 - * :
 - * : $(x \times y) \times z = x \times (y \times z)$. (Idem).
 - * : $x \times y = y \times x$. (Idem).
 - * : La fraction $1/1$ (soit `Fraction(1, 1)`) est l'élément neutre de la multiplication, car $x \times 1/1 = x$.

* : Chaque fraction $a/b \neq 0/1$ a un inverse b/a (utilisé dans `__truediv__`) tel que $a/b \times b/a = 1/1$.

* : $x \times (y + z) = (x \times y) + (x \times z)$. Cette propriété découle directement de la distributivité des entiers et de la définition de nos opérations sur les fractions.

2. $\leq$ est un ordre total sur $\mathbb{Q}$:

* : $x \leq x$ (implémenté par `__le__` et `__eq__`).

* : Si $x \leq y$ et $y \leq x$, alors $x = y$ (implémenté par `__le__` et `__eq__`).

* : Si $x \leq y$ et $y \leq z$, alors $x \leq z$. La comparaison $a/b < c/d \iff ad < bc$ maintient cette propriété due à la transitivité de $<$ sur les entiers.

* : Pour tout $x, y \in \mathbb{Q}$, soit $x \leq y$ soit $y \leq x$. Cela est vrai pour notre implémentation car $ad < bc$, $ad = bc$, ou $ad > bc$ sont les seules possibilités pour les entiers ad et bc .

3. Compatibilité de l'ordre avec les opérations :

* : Si $x \leq y$, alors $x + z \leq y + z$. Soit $x = a/b$, $y = c/d$, $z = e/f$. Si $ad \leq bc$, nous voulons montrer que $af/bf + be/bf \leq cf/df + de/df$, i.e. $(af + be)/(bf) \leq (cf + de)/(df)$. En multipliant les numérateurs par les dénominateurs respectifs on compare $(af + be)df$ et $(cf + de)bf$. Cela découle des propriétés des entiers et du fait que les dénominateurs sont positifs.

* : Si $x \leq y$ et $z > 0$, alors $x \times z \leq y \times z$. Si $a/b \leq c/d$ et $e/f > 0$ (donc $e > 0, f > 0$), nous voulons montrer que $(a/b)(e/f) \leq (c/d)(e/f)$, i.e. $ae/bf \leq ce/df$. Puisque e, f, b, d sont positifs, on compare $aedf$ et $bcef$. Si $ad \leq bc$, alors $ad \cdot ef \leq bc \cdot ef$. Puisque $e, f > 0$, $ef > 0$. Donc $aedf \leq bcef$.

En conclusion, notre implémentation de la classe `Fraction` respecte bien les axiomes d'un corps ordonné, ce qui fait de l'ensemble des instances de `Fraction` (conceptuellement, $\mathbb{Q}$) un corps ordonné.

Conclusion du TP-01

Nous avons construit avec succès un système pour représenter et manipuler les nombres rationnels en Python. Ce système satisfait les propriétés d'un corps ordonné. Cependant, comme nous le verrons dans le prochain TP, cette structure, bien que riche, n'est pas suffisante pour l'analyse mathématique, car elle manque de la propriété de complétude, c'est-à-dire l'axiome de la borne supérieure.

'yaml

TP-02 : Bornes et Incomplétude des Nombres Rationnels

Introduction

Après avoir établi que les nombres rationnels ($\mathbb{Q}$) forment un corps ordonné dans le TP-01, ce TP-02 se concentre sur les notions de bornes et, plus crucialement, sur l'incomplétude de $\mathbb{Q}$. L'incomplétude est la raison fondamentale pour laquelle les nombres réels ($\mathbb{R}$) ont été introduits. Nous démontrerons que certains ensembles de nombres rationnels, bien que bornés supérieurement, n'admettent pas de borne supérieure (supremum) qui soit elle-même un nombre rationnel.

Prérequis Mathématiques

Définition 1 (Ensemble borné supérieurement) : Un ensemble S de nombres est dit borné supérieurement s'il existe un nombre M tel que pour tout $x \in S$, $x \leq M$. Un tel M est appelé une borne supérieure de S .

Définition 2 (Supremum / Borne supérieure) : Le supremum (ou la borne supérieure) d'un ensemble S , noté $\sup(S)$, est la plus petite de toutes les bornes supérieures de S . Si $\sup(S)$ existe, il doit satisfaire deux conditions : 1. $\sup(S)$ est une borne supérieure de S (pour tout $x \in S$, $x \leq \sup(S)$). 2. Pour toute borne supérieure M' de S , $\sup(S) \leq M'$. (Ou de manière équivalente : pour tout $\epsilon > 0$, il existe $x \in S$ tel que $\sup(S) - \epsilon < x$).

Définition 3 (Ensemble borné inférieurement) : Un ensemble S de nombres est dit borné inférieurement s'il existe un nombre m tel que pour tout $x \in S$, $x \geq m$. Un tel m est appelé une borne inférieure de S .

Définition 4 (Infimum / Borne inférieure) : L'infimum (ou la borne inférieure) d'un ensemble S , noté $\inf(S)$, est la plus grande de toutes les bornes inférieures de S .

Axiome de la Borne Supérieure (ou Axiome de Complétude de $\mathbb{R}$) : Tout ensemble non vide de nombres réels qui est borné supérieurement admet une borne supérieure dans $\mathbb{R}$. Cet axiome est ce qui distingue $\mathbb{R}$ de $\mathbb{Q}$.

Exercice 1 : Bornes dans $\mathbb{Q}$

Nous allons réutiliser la classe `Fraction` du TP-01.

Tâche 1.1 : Fonctions pour les bornes

Implémentez les fonctions suivantes pour un ensemble de `Fraction` : `* est_borne_superieurement(S)` : retourne `True` si S est borné supérieurement, `False` sinon. `* trouver_une_borne_superieure(S)` : retourne une borne supérieure de S si S est borné supérieurement, sinon lève une exception ou retourne `None`. s'il est un rationnel*. Retourne ce supremum sous forme de `Fraction` si trouvé avec une certaine `tolerance` (pour des comparaisons numériques), sinon `None`.

```
# Re-utiliser la classe Fraction du TP-01
# Copier-coller le code de Fraction et gcd ici pour l'autonomie du fichier
def gcd(a, b):
    a_abs = abs(a)
    b_abs = abs(b)
    while b_abs:
        a_abs, b_abs = b_abs, a_abs % b_abs
    return a_abs

class Fraction:
    def __init__(self, numérateur, dénominateur):
```

```

    if denominateur == 0:
        raise ValueError("Le denominateur ne peut pas etre zero.")
    if denominateur < 0:
        numerateur = -numerateur
        denominateur = -denominateur
    common_divisor = gcd(numerateur, denominateur)
    self.numerateur = numerateur // common_divisor
    self.denominateur = denominateur // common_divisor

def __str__(self):
    if self.denominateur == 1:
        return str(self.numerateur)
    return f"{self.numerateur}/{self.denominateur}"

def __repr__(self):
    return f"Fraction({self.numerateur}, {self.denominateur})"

def __eq__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return self.numerateur == other.numerateur and self.denominateur == other.denominateur

def __lt__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return self.numerateur
self.denominateur

def __le__(self, other):
    return self < other or self == other

def __gt__(self, other):
    return not (self <= other)

def __ge__(self, other):
    return not (self < other)

def __add__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    new_numerateur = self.numerateur
self.denominateur
    new_denominateur = self.denominateur * other.denominateur
    return Fraction(new_numerateur, new_denominateur)

def __radd__(self, other):
    return self.__add__(other)

def __sub__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    new_numerateur = self.numerateur
self.denominateur
    new_denominateur = self.denominateur * other.denominateur
    return Fraction(new_numerateur, new_denominateur)

def __rsub__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return Fraction(other, 1) - self

```



```

def __mul__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    new_numérateur = self.numérateur * other.numérateur
    new_dénominateur = self.dénominateur * other.dénominateur
    return Fraction(new_numérateur, new_dénominateur)

def __rmul__(self, other):
    return self.__mul__(other)

def __truediv__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    if other.numérateur == 0:
        raise ZeroDivisionError("Division par zero.")
    return self * Fraction(other.dénominateur, other.numérateur)

def __rtruediv__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return Fraction(other, 1) / self

def __neg__(self):
    return Fraction(-self.numérateur, self.dénominateur)

def __abs__(self):
    return Fraction(abs(self.numérateur), self.dénominateur)

def est_borne_supérieurement(S):
    """
    Vérifie si un ensemble S de Fractions est borne supérieure.
    Un ensemble est borne supérieure s'il est vide ou si un majorant existe.
    """
    if not S:
        return True # Conventionnellement, l'ensemble vide est borne
    # Si S est non vide et contient au moins un élément, il faut vérifier l'existence
    # d'un majorant.
    # Pour un ensemble fini, le maximum est une borne supérieure.
    # Pour un ensemble infini, nous ne pouvons pas le déterminer sans information
    # supplémentaire sur sa construction.
    # Pour les besoins de ce TP, nous allons considérer des ensembles finis ou des
    # ensembles définis par des propriétés
    # pour lesquels un majorant est conceptuellement clair.
    # Dans le cadre de ce TP, nous considérerons S comme une liste Python, donc finie
    return True # Tout ensemble fini de nombres est borne.

def trouver_une_borne_supérieure(S):
    """
    Trouve une borne supérieure pour un ensemble S de Fractions.
    Pour un ensemble fini, c'est simplement le maximum.
    """
    if not S:
        raise ValueError("L'ensemble vide n'a pas de borne supérieure concrète (la
            notion est triviale).")

    max_val = S[0]
    for x in S:
        if x > max_val:
            max_val = x
    return max_val

```

```

def trouver_supremum_dans_Q(S, tolerance_n=1, tolerance_d=1000):
    """
    Tente de trouver le supremum d'un ensemble S de Fractions dans Q.
    Cette fonction est une heuristique pour des ensembles "simples".
    Pour un ensemble S, si son supremum est dans Q, il doit etre le maximum si S
        contient son supremum.
    Sinon, on doit chercher la plus petite borne superieure.
    Ici, nous allons chercher une borne superieure rationnelle potentielle
    qui soit "proche" de la borne que nous conjecturons.
    Pour les ensembles de type {x in Q | x < M}, le supremum est M si M est dans Q.
    Pour les ensembles de type {x in Q | x <= M}, le supremum est M si M est dans Q.
    Nous allons simplifier et chercher le maximum pour des ensembles finis.
    Pour des ensembles infinis definis par une regle, c'est plus complexe.

    Nous allons implementer une recherche pour un ensemble  $S = \{x \in \mathbb{Q} \mid$ 
         $x^2 < k\}$ 
    ou  $k$  est un rationnel. On cherche  $\sqrt{k}$ .

    'tolerance_n' et 'tolerance_d' definissent une "precision" pour la recherche,
    signifiant que l'on cherche un rationnel  $p/q$  tel que  $p^2/q^2$  est tres proche
    de  $k$ .
    """
    if not S:
        return None

    # Si S est simplement une liste finie, le supremum est le maximum
    if all(isinstance(x, Fraction) for x in S):
        return trouver_une_borne_superieure(S)

    # Cas specifique pour  $S = \{x \in \mathbb{Q} \mid x^2 < K\}$  pour K rationnel
    # Cette partie est une tentative d'implementation pour l'incompletude.
    # Elle ne peut pas "prouver" un supremum non-Q, mais peut montrer l'echec de
    # trouver un Q-supremum.
    # Ici, nous allons
    que S est de la forme  $S = \{x \in \mathbb{Q} \mid x^2 < K\}$ , ou K est un nombre rationnel.
    # On cherche le supremum de S, qui est  $\sqrt{K}$ .
    # Si  $\sqrt{K}$  n'est pas rationnel, cette fonction ne trouvera pas de supremum DANS
    # Q.
    # On va rechercher un candidat  $p/q$  tel que  $(p/q)^2$  soit tres proche de K.
    # Nous allons prendre le max des elements de S comme point de depart

    # Pour simuler la recherche d'un supremum pour un ensemble infini  $\{x \in \mathbb{Q} \mid P(x)$ 
    #  $\}$ ,
    # nous devons nous restreindre a des cas ou  $P(x)$  est testable.
    # La maniere la plus simple d'illustrer l'incompletude est de prendre un ensemble
    # dont le supremum est clairement irrationnel.
    # Par exemple,  $S = \{x \in \mathbb{Q} \mid x^2 < 2\}$ . Le supremum est  $\sqrt{2}$ .
    # Dans ce cas, on peut chercher des rationnels  $p/q$  tels que  $(p/q)^2$  se rapproche
    # de 2.
    # La fonction doit retourner le "candidat" supremum si  $(candidat^2 == K)$ .

    # Pour ce TP, nous allons utiliser une approche plus simple :
    # Si le supremum est dans S, c'est le max. Sinon, il n'est pas "trivialement"
    # trouvable.
    # Une implementation robuste de 'trouver_supremum_dans_Q' necessiterait une
    # definition plus precise de S.
    # Nous allons simuler la recherche du supremum pour un ensemble fini, qui sera
    # son maximum.
    # Si le supremum d'un ensemble infini est demande, nous allons plutot
    si un certain candidat
    est le supremum.

    # Pour l'exemple  $\{x \in \mathbb{Q} \mid x^2 < 2\}$ , on peut iterer pour trouver le plus grand
    # rationnel
    # dont le carre est < 2.
    # Initialisation arbitraire.

```

```

max_in_S = Fraction(0, 1)

# Si l'ensemble S est fourni comme une liste de Fractions, on prend le maximum.
if all(isinstance(val, Fraction) for val in S):
    return trouver_une_borne_superieure(S)

# Autre cas : S defini par une propriete, par exemple {x in Q | x*x < k}.
# Nous n'avons pas de moyen general de simuler un ensemble infini de Q dans
    Python.
# Nous allons simuler un ensemble fini de candidats et chercher le plus grand.
# Cela
que le supremum est en Q, mais illustre une recherche.

# Pour l'illustration de l'incompletude avec sqrt(2), nous allons considerer
# un ensemble S_sqrt2 = {x in Q | x^2 < 2 et x > 0}.
# Cette fonction ne peut pas "generer" tous les elements de S_sqrt2.
# Elle peut en revanche tester si un CANDIDAT rationnel est le supremum.

# Par convention pour cet exercice, 'S' est soit une liste de Fractions, soit un
    identifiant symbolique.
# Si c'est une liste, on retourne le max. Si c'est pour l'incompletude, on
    retourne None.
print("ATTENTION: 'trouver_supremum_dans_Q' est une heuristique simplifiee pour
    ce TP.")
print("Elle ne peut pas prouver l'existence d'un supremum pour des ensembles
    infinis.")
return None # Par default, si non trivial, on ne trouve pas de supremum dans Q.

# Exemple d'utilisation et Assertions
# Ensemble 1 : S1 = {x in Q | x < 2} (approx.)
# On simule S1 avec des fractions proches de 2. Le supremum est 2 (dans Q).
S1 = [Fraction(1, 1), Fraction(3, 2), Fraction(7, 4), Fraction(15, 8), Fraction(1.999
    * 1000, 1000)] # pour simuler des fractions arbitraires < 2
# Filtrons les elements de S1 pour qu'ils soient rationnels purs
S1 = [Fraction(1,1), Fraction(3,2), Fraction(7,4), Fraction(199,100), Fraction
    (1999,1000)]
print(f"Ensemble S1: {[str(f) for f in S1]}")
assert est_borne_superieurement(S1) == True, "S1 devrait etre borne superieurement."
borne_sup_S1 = trouver_une_borne_superieure(S1)
print(f"Une borne superieure pour S1 (le max de la liste) est : {borne_sup_S1}")
# Le vrai supremum de {x in Q | x < 2} est 2. Notre fonction est heuristique ici.
# Si nous voulons verifier si Fraction(2,1) est le supremum:
candidat_sup_S1 = Fraction(2, 1)
# Verifions si le candidat est une borne superieure pour TOUS les elements de S1
is_sup_candidate_S1 = all(x <= candidat_sup_S1 for x in S1)
# Verifions si c'est la PLUS PETITE borne superieure (difficile a tester
    generiquement)
# Pour une liste finie, le max est le supremum.
assert borne_sup_S1 == Fraction(1999, 1000), "Le max de S1 devrait etre 1999/1000"
print(f"Le supremum
    est {borne_sup_S1}.")

# Ensemble 2 : S2 = {x in Q | x^2 < 2 et x > 0}
# Le supremum de S2 est sqrt(2), qui n'est pas dans Q.
# Nous allons generer quelques fractions dont le carre est < 2.
S2_elements = []
max_q_val = 0 # pour suivre le plus grand rationnel trouve
# Simulation d'une recherche limitee de rationnels
# On explore les fractions p/q avec q allant jusqu'a un certain Dmax
D_MAX = 500 # Denominateur maximum pour la recherche limitee
for q in range(1, D_MAX + 1):
    for p in range(1, 2
q/q = 2, car sqrt(2) < 2
        f_cand = Fraction(p, q)
        if f_cand * f_cand < Fraction(2, 1):

```

```

        S2_elements.append(f_cand)

# Trouver la plus grande fraction dans S2_elements
if S2_elements:
    S2_approx_sup = trouver_une_borne_superieure(S2_elements)
else:
    S2_approx_sup = None

print(f"\nExemple d'elements dans S2_elements (quelques uns): {[str(f) for f in
    S2_elements[:10]]} ...")
print(f"Plus grande fraction trouvee dans S2_elements (approx. du supremum) : {
    S2_approx_sup}")

# Assertions Mathematiques pour l'incompletude
# 1. S2 est borne superieurement.
assert est_borne_superieurement(S2_elements) == True, "S2_elements devrait etre borne
    superieurement."
print("S2_elements est borne superieurement (par exemple par 2/1).")

# 2. Le supremum de S2_elements (conceptuellement S2) n'est pas dans Q.
# Nous ne pouvons pas
    cette non-existence avec Python pur directement.
# Nous pouvons seulement montrer que notre 'trouver_supremum_dans_Q' ne le trouve pas
    si nous l'avions implemente pour ca.
# Ici, nous affirmons mathematiquement :
print("\nValidation Algorithmique Rigoureuse et Assertions Mathematiques de l'
    incompletude de Q :")
print("Le supremum de l'ensemble  $S_2 = \{x \in \mathbb{Q} \mid x^2 < 2 \text{ et } x > 0\}$  est  $\sqrt{2}$  est")
print("Par demonstration classique (demontrée par les Grecs anciens),  $\sqrt{2}$  est
    un nombre irrationnel, c'est-a-dire  $\sqrt{2} \notin \mathbb{Q}$  est")
print("Par consequent, l'ensemble  $S_2$  est borne superieurement dans  $\mathbb{Q}$  (
    par exemple, par 2), mais il n'admet pas de borne superieure
     $\mathbb{Q}$  est.")
print("Ceci demontre que  $\mathbb{Q}$  n'est pas un corps complet au sens de l'axiome
    de la borne superieure.")
print(f"Notre recherche limitee avec D_MAX={D_MAX} a trouve {S2_approx_sup} comme le
    plus grand rationnel  $p/q$  avec  $q \leq \{D\_MAX\}$  tel que  $(p/q)^2 < 2$  est.")
print(f"Son carre est : {(S2_approx_sup * S2_approx_sup)}") # devrait etre proche de
    2, mais < 2
# Le prochain rationnel au-dessus S2_approx_sup pourrait avoir un carre > 2
# Ou bien le prochain rationnel  $p'/q'$  tel que  $(p'/q')^2 < 2$  pourrait avoir un  $q' >
    D\_MAX$ 

# Un autre exemple simple : si on cherche le supremum de {1, 2, 3}. Le supremum est
    3, qui est dans Q.
S_simple = [Fraction(1,1), Fraction(2,1), Fraction(3,1)]
sup_S_simple = trouver_supremum_dans_Q(S_simple)
assert sup_S_simple == Fraction(3,1), "Supremum pour ensemble simple devrait etre 3/1"
print(f"Supremum de {{1, 2, 3}} : {sup_S_simple}")

# Test de la non-existence avec notre fonction simplifiee (elle renvoie None pour des
    cas complexes)
assert trouver_supremum_dans_Q(S2_elements) is None, "La fonction ne devrait pas
    trouver de supremum dans Q pour S2_elements (car le vrai est irrationnel)."
print("La fonction 'trouver_supremum_dans_Q' renvoie 'None' pour des ensembles comme
    S2_elements, indiquant l'echec de trouver un supremum
    evident.")

```

Conclusion du TP-02

Dans ce TP, nous avons exploré les concepts de borne supérieure pour des ensembles de nombres rationnels. Nous avons surtout mis en évidence l'incomplétude de $\mathbb{Q}$. Nous avons illustré, à travers l'exemple de l'ensemble

$S_2 = \{x \in \mathbb{Q} \mid x^2 < 2 \text{ et } x > 0\}$, qu'un ensemble borné supérieurement dans $\mathbb{Q}$ peut ne pas posséder de borne supérieure appartenant à $\mathbb{Q}$. Ce "trou" dans la droite numérique rationnelle est précisément ce qui est comblé par l'introduction des nombres irrationnels et l'axiome de la borne supérieure, qui est le fondement de la construction des nombres réels $\mathbb{R}$. Le prochain TP abordera une autre propriété fondamentale des réels : la propriété d'Archimède.

'yaml

TP-03 : La Propriété d'Archimède

Introduction

La propriété d'Archimède est un axiome fondamental qui caractérise les nombres réels et rationnels comme des corps ordonnés "archimédiens". Elle exprime intuitivement qu'il n'existe pas de nombres "infiniment grands" ou "infiniment petits" au sens où tout nombre, aussi grand soit-il, peut être dépassé par la somme répétée d'un autre nombre positif, aussi petit soit-il. Cette propriété est cruciale pour l'analyse, notamment pour la notion de convergence et la densité des rationnels dans les réels.

Prérequis Mathématiques

Définition 1 (Nombres Réels - Représentation pratique) : Pour ce TP, nous utiliserons les nombres à virgule flottante (`float`) de Python pour représenter les nombres réels. Il est essentiel de noter que les `float` sont une approximation finie des nombres réels, avec des limitations de précision. Néanmoins, pour des illustrations pratiques de la propriété d'Archimède, ils sont suffisants.

Propriété d'Archimède (pour $\mathbb{R}$) : Pour tous nombres réels x et y tels que $x > 0$, il existe un entier naturel n ($n \in \mathbb{N}$) tel que $n \cdot x > y$.

Conséquences directes de la Propriété d'Archimède : 1. Les entiers naturels sont non bornés supérieurement dans $\mathbb{R}$: Si on prend $x = 1$ et y un réel arbitraire, il existe $n \in \mathbb{N}$ tel que $n \cdot 1 > y$. Cela signifie que pour tout réel y , il existe un entier naturel plus grand que y . 2. Pour tout $\epsilon > 0$, il existe un entier n tel que $1/n < \epsilon$: Si on prend $y = 1$ et $x = \epsilon$ dans la propriété d'Archimède, il existe $n \in \mathbb{N}$ tel que $n \cdot \epsilon > 1$. En divisant par $n \cdot \epsilon$ (qui est positif), on obtient $1/n < \epsilon$. Cela signifie que des intervalles arbitrairement petits autour de 0 peuvent être construits en utilisant des inverses d'entiers naturels. 3. Pour tout $x \in \mathbb{R}$, il existe un entier n tel que $n \leq x < n + 1$ (partie entière).

Exercice 1 : Vérification de la Propriété d'Archimède

Tâche 1.1 : Fonction `verifier_archimede`

Implémentez la fonction `verifier_archimede(x, y)` qui prend deux nombres réels x (strictement positif) et y (arbitraire) et retourne le plus petit entier naturel n tel que $n \cdot x > y$. Si $x \leq 0$, la fonction doit lever une `ValueError`.

```
def verifier_archimede(x, y):
    """
    Verifie la propriete d'Archimede pour x et y.
    Trouve le plus petit entier naturel n tel que n * x > y.

    Arguments:
        x (float): Un nombre reel strictement positif.
        y (float): Un nombre reel.

    Retourne:
        int: Le plus petit entier naturel n satisfaisant n * x > y.

    Leve:
        ValueError: Si x n'est pas strictement positif.
    """
    if not isinstance(x, (float, int)) or not isinstance(y, (float, int)):
        raise TypeError("x et y doivent etre des nombres (float ou int).")
    if x <= 0:
        raise ValueError("Le nombre x doit etre strictement positif.")
```

```

n = 0
# On commence la recherche a 0.  $n \cdot x > y$ 
# Si y est negatif, n=0 ou n=1 sera souvent suffisant (0
x=x).
# n doit etre un entier naturel, donc  $n \geq 0$ .

# Strategie : Demarrer n=1 si  $y \geq 0$ .
# Si  $y < 0$ , n=0 peut suffire si  $0 > y$ .
# Si  $x > y$ , n=1 suffit.

# Une approche simple et robuste est d'incrémenter n a partir de 0 jusqu'a ce que
# la condition soit remplie.
# Puisque nous cherchons le PLUS PETIT entier n, commencer a 0 et incrémenter est
# correct.
while n * x <= y:
    n += 1
return n

# Assertions Mathematiques pour verifier_archimede
# Cas 1:  $x > 0$ ,  $y > 0$ 
#  $x=0.5$ ,  $y=3.0$ . On cherche n tel que  $n \cdot 0.5 > 3.0 \Rightarrow n > 6$ . Le plus petit n est 7.
assert verifier_archimede(0.5, 3.0) == 7, "Assertion echouee : verifier_archimede
(0.5, 3.0)"
#  $x=10$ ,  $y=3.0$ . On cherche n tel que  $n \cdot 10 > 3.0 \Rightarrow n > 0.3$ . Le plus petit n est 1.
assert verifier_archimede(10.0, 3.0) == 1, "Assertion echouee : verifier_archimede
(10.0, 3.0)"
# Cas 2:  $x > 0$ ,  $y \leq 0$ 
#  $x=5.0$ ,  $y=-2.0$ . On cherche n tel que  $n \cdot 5.0 > -2.0$ . Pour  $n=0$ ,  $0 > -2.0$ . Le plus
# petit n est 0.
assert verifier_archimede(5.0, -2.0) == 0, "Assertion echouee : verifier_archimede
(5.0, -2.0)"
# Cas 3:  $x > 0$ ,  $y = 0$ 
#  $x=3.0$ ,  $y=0.0$ . On cherche n tel que  $n \cdot 3.0 > 0.0$ . Pour  $n=1$ ,  $3.0 > 0.0$ . Le plus
# petit n est 1.
assert verifier_archimede(3.0, 0.0) == 1, "Assertion echouee : verifier_archimede
(3.0, 0.0)"
# Cas 4: x est un entier, y est un entier
assert verifier_archimede(2, 7) == 4, "Assertion echouee : verifier_archimede(2, 7)"
# Cas 5: x et y grands
assert verifier_archimede(1e-6, 1e6) == 10000000000001, "Assertion echouee :
verifier_archimede(1e-6, 1e6)"
# Cas d'erreur :  $x \leq 0$ 
try:
    verifier_archimede(-1.0, 5.0)
    assert False, "Assertion echouee : ValueError attendue pour  $x \leq 0$ "
except ValueError:
    assert True, "ValueError correctement levee pour  $x \leq 0$ "
try:
    verifier_archimede(0.0, 5.0)
    assert False, "Assertion echouee : ValueError attendue pour  $x \leq 0$ "
except ValueError:
    assert True, "ValueError correctement levee pour  $x \leq 0$ "

print("Toutes les assertions pour 'verifier_archimede' ont reussi.")

```

Exemples d'Illustration de la Propriété d'Archimède

```

print("\n--- Illustrations de la Propriete d'Archimede ---")

# Illustration 1 : Les entiers naturels sont non bornes superieurement.
# Pour tout y reel, il existe un entier n tel que  $n > y$ .
y_val = 1_000_000_000_000.0 # Un tres grand nombre
n_archimede = verifier_archimede(1.0, y_val) # Ici  $x=1$ 

```



```

print(f"Pour y = {y_val}, le plus petit entier n tel que  $n * 1.0 > y$  est  $n = \{n\_archimede\}$ .").)
print(f"    Cela illustre que les entiers naturels peuvent dépasser n'importe quel nombre reel y.").)
assert n_archimede > y_val, "L'illustration 1 a echoue."

# Illustration 2 : Pour tout epsilon > 0, il existe n tel que  $1/n < \epsilon$ .
# Ceci implique que des nombres arbitrairement petits peuvent etre formes.
def trouver_n_pour_epsilon(epsilon):
    """
    Trouve le plus petit entier naturel n tel que  $1/n < \epsilon$ .
    Ceci est equivalent a  $n > 1/\epsilon$ .
    """
    if not isinstance(epsilon, (float, int)):
        raise TypeError("epsilon doit etre un nombre.")
    if epsilon <= 0:
        raise ValueError("epsilon doit etre strictement positif.")

    # Utiliser la propriete d'Archimede :  $n * \epsilon > 1$ 
    # Donc  $n > 1 / \epsilon$ 
    # Le plus petit entier n est  $\text{floor}(1/\epsilon) + 1$  si  $1/\epsilon$  n'est pas entier
    # ou  $1/\epsilon + 1$  si  $1/\epsilon$  est entier
    # La fonction verifier_archimede(x=epsilon, y=1) nous donne directement n tel que
    #  $n * \epsilon > 1$ .
    return verifier_archimede(epsilon, 1.0)

epsilon_val = 1e-9 # Un tres petit nombre positif
n_epsilon = trouver_n_pour_epsilon(epsilon_val)
print(f"\nPour epsilon = {epsilon_val}, le plus petit entier n tel que  $1/n < \epsilon$  est  $n = \{n\_epsilon\}$ .").)
print(f"    Verification :  $1/\{n\_epsilon\} = \{1/n\_epsilon\}$ , qui est bien  $< \{epsilon\_val\}$ .").)
assert 1 / n_epsilon < epsilon_val, "L'illustration 2 a echoue."
print(f"    Cela illustre qu'il n'y a pas de 'plus petit reel positif' (infinitesimal non nul).")

# Illustration 3 : Approximation d'un reel par des rationnels
# La propriete d'Archimede est utilisee pour montrer la densite des rationnels dans les reels.
# Pour tout reel x, il existe un entier n tel que  $n \leq x < n+1$  (fonction partie entiere).
# Cela permet de borner x entre deux entiers consecutifs.
def partie_entiere(x):
    """
    Retourne l'entier n tel que  $n \leq x < n+1$ .
    """
    if not isinstance(x, (float, int)):
        raise TypeError("x doit etre un nombre.")

    # Pour  $x \geq 0$ , la fonction int(x) fonctionne.
    # Pour  $x < 0$ , int(x) tronque vers zero, ce qui n'est pas la partie entiere (floor).
    # Exemple:  $\text{int}(-3.7) = -3$ , mais  $\text{floor}(-3.7) = -4$ .
    # Python's math.floor() existe mais nous devons l'implementer nous-memes.
    if x >= 0:
        return int(x)
    else:
        # Pour les negatifs, trouver l'entier n tel que  $n \leq x < n+1$ 
        # Si  $x = -3.7$ ,  $n = -4$ .
        # Si int(x) est -3, et x n'est pas exactement -3.0, alors n doit etre int(x) - 1.
        if x == int(x):
            return int(x)
        else:
            return int(x) - 1 # floor pour les negatifs non entiers

```

```

x_val = 3.14159
n_partie_entiere = partie_entiere(x_val)
print(f"\nPour x = {x_val}, la partie entiere est n = {n_partie_entiere}.")
print(f"  Verification : {n_partie_entiere} <= {x_val} < {n_partie_entiere + 1}.")
assert n_partie_entiere <= x_val < n_partie_entiere + 1, "L'illustration 3 (positif)
    a echoue."

x_val_neg = -2.718
n_partie_entiere_neg = partie_entiere(x_val_neg)
print(f"Pour x = {x_val_neg}, la partie entiere est n = {n_partie_entiere_neg}.")
print(f"  Verification : {n_partie_entiere_neg} <= {x_val_neg} < {
    n_partie_entiere_neg + 1}.")
assert n_partie_entiere_neg <= x_val_neg < n_partie_entiere_neg + 1, "L'illustration
    3 (negatif) a echoue."

print(f"  Cela illustre la propriete d'Archimede permettant de borner tout reel entre
    deux entiers consecutifs,")
print(f"  une etape cruciale pour la construction des rationnels et des reels.")

```

Validation Algorithmique Rigoureuse et Assertions Mathématiques : La Propriété d'Archimède comme fondement.

La propriété d'Archimède n'est pas dérivable des axiomes d'un corps ordonné seul (un corps ordonné non archimédien, comme les nombres hyperréels, existe). Elle est un axiome supplémentaire nécessaire pour caractériser les réels.

Notre implémentation `verifier_archimede(x, y)` démontre empiriquement l'existence de l'entier n . Pour un $x > 0$ et un y donnés : 1. Si $y \leq 0$, alors pour $n = 0$, $0 \cdot x = 0$. Si $0 > y$, alors $n = 0$ est le plus petit entier. Si $y = 0$, $0 \cdot x \not> 0$, donc $n = 1$ est nécessaire (si $x > 0$, $1 \cdot x > 0$). Notre boucle `while n * x <= y` capture ceci correctement. 2. Si $y > 0$, l'existence de n tel que $n \cdot x > y$ est une conséquence directe de la propriété. Le fait que n puisse être trouvé par incrémentation successive (même si inefficace pour de très grands nombres, c'est conceptuellement juste) confirme cette existence. La boucle terminera toujours, car si elle ne terminait pas, cela signifierait que pour tout n , $n \cdot x \leq y$, ce qui voudrait dire que l'ensemble $\{n \cdot x \mid n \in \mathbb{N}^*\}$ est borné supérieurement par y/x , ce qui contredit la première conséquence de la propriété d'Archimède elle-même (Les entiers naturels sont non bornés supérieurement dans $\mathbb{R}$).

Ces démonstrations par l'exemple confirment le rôle essentiel de la propriété d'Archimède : elle assure que la droite numérique n'a pas de "trous" dus à des nombres infiniment grands ou petits, et elle prépare le terrain pour la densité des rationnels dans les réels.

Conclusion du TP-03

Ce TP a mis en lumière la propriété d'Archimède, un principe fondamental des nombres réels. Nous avons implémenté une fonction pour la vérifier et illustré ses conséquences directes : l'infinité des entiers naturels et la possibilité de trouver des nombres arbitrairement petits sous forme de $1/n$. Ces concepts sont essentiels pour la compréhension de la structure de $\mathbb{R}$ et seront réutilisés dans les prochains TPs, notamment pour aborder la densité de $\mathbb{Q}$ dans $\mathbb{R}$ en conjonction avec l'axiome de la borne supérieure.

'yaml

TP-04 : Approximation des Nombres Réels et Séquences

Introduction

Dans le TP-02, nous avons constaté l'incomplétude des nombres rationnels ($\mathbb{Q}$) : certains ensembles bornés supérieurement n'y ont pas de supremum. L'axiome de la borne supérieure garantit l'existence de ces suprema dans les nombres réels ($\mathbb{R}$). Or, beaucoup de ces suprema sont des nombres irrationnels. Ce TP se propose d'explorer comment nous pouvons "construire" ou du moins ces nombres irrationnels à l'aide de séquences de nombres rationnels. C'est une approche intuitive pour comprendre comment les irrationnels "remplissent les trous" de la droite rationnelle.

Prérequis Mathématiques

Définition 1 (Suite de Cauchy) : Une suite de nombres réels $(x_n)_{n \in \mathbb{N}}$ est dite de Cauchy si pour tout $\epsilon > 0$, il existe un entier N tel que pour tous $m, n > N$, on a $|x_m - x_n| < \epsilon$. Intuitivement, les termes d'une suite de Cauchy se rapprochent arbitrairement les uns des autres à mesure que n augmente.

Définition 2 (Convergence d'une suite) : Une suite $(x_n)_{n \in \mathbb{N}}$ converge vers une limite L si pour tout $\epsilon > 0$, il existe un entier N tel que pour tout $n > N$, on a $|x_n - L| < \epsilon$.

Théorème (Complétude de $\mathbb{R}$ via suites de Cauchy) : Dans $\mathbb{R}$, toute suite de Cauchy converge vers un nombre réel. Ce théorème est équivalent à l'axiome de la borne supérieure : si l'un est supposé, l'autre peut être prouvé. $\mathbb{Q}$ n'est pas complet car une suite de Cauchy de rationnels peut converger vers un irrationnel.

Exercice 1 : Approximation de $\sqrt{2}$ par la méthode de Héron d'Alexandrie (Newton-Raphson)

La méthode de Héron d'Alexandrie (souvent appelée méthode de Newton-Raphson pour la recherche de racines carrées) est un algorithme itératif pour trouver des approximations de racines carrées. Pour calculer $\sqrt{K}$, la suite est définie par $x_{n+1} = \frac{1}{2} \left(x_n + \frac{K}{x_n} \right)$.

Tâche 1.1 : Générateur de suite d'approximation de $\sqrt{K}$

Implémentez une fonction `heron_sqrt(K, initial_guess, num_iterations)` qui génère une liste des premières `num_iterations` approximations de $\sqrt{K}$ sous forme de `Fraction`.

```
# Re-utiliser la classe Fraction du TP-01
# Copier-coller le code de Fraction et gcd ici pour l'autonomie du fichier
def gcd(a, b):
    a_abs = abs(a)
    b_abs = abs(b)
    while b_abs:
        a_abs, b_abs = b_abs, a_abs % b_abs
    return a_abs

class Fraction:
    def __init__(self, numérateur, dénominateur):
        if dénominateur == 0:
            raise ValueError("Le dénominateur ne peut pas être zero.")
        if dénominateur < 0:
```

```

        numerateur = -numerateur
        denominateur = -denominateur
        common_divisor = gcd(numerateur, denominateur)
        self.numerateur = numerateur // common_divisor
        self.denominateur = denominateur // common_divisor

    def __str__(self):
        if self.denominateur == 1:
            return str(self.numerateur)
        return f"{self.numerateur}/{self.denominateur}"

    def __repr__(self):
        return f"Fraction({self.numerateur}, {self.denominateur})"

    def __eq__(self, other):
        if not isinstance(other, Fraction):
            if isinstance(other, int): other = Fraction(other, 1)
            else: return NotImplemented
        return self.numerateur == other.numerateur and self.denominateur == other.denominateur

    def __lt__(self, other):
        if not isinstance(other, Fraction):
            if isinstance(other, int): other = Fraction(other, 1)
            else: return NotImplemented
        return self.numerateur
self.denominateur

    def __le__(self, other):
        return self < other or self == other

    def __gt__(self, other):
        return not (self <= other)

    def __ge__(self, other):
        return not (self < other)

    def __add__(self, other):
        if not isinstance(other, Fraction):
            if isinstance(other, int): other = Fraction(other, 1)
            else: return NotImplemented
        new_numerateur = self.numerateur
self.denominateur
        new_denominateur = self.denominateur * other.denominateur
        return Fraction(new_numerateur, new_denominateur)

    def __radd__(self, other):
        return self.__add__(other)

    def __sub__(self, other):
        if not isinstance(other, Fraction):
            if isinstance(other, int): other = Fraction(other, 1)
            else: return NotImplemented
        new_numerateur = self.numerateur
self.denominateur
        new_denominateur = self.denominateur * other.denominateur
        return Fraction(new_numerateur, new_denominateur)

    def __rsub__(self, other):
        if not isinstance(other, Fraction):
            if isinstance(other, int): other = Fraction(other, 1)
            else: return NotImplemented
        return Fraction(other, 1) - self

    def __mul__(self, other):
        if not isinstance(other, Fraction):

```

```

        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    new_numérateur = self.numérateur * other.numérateur
    new_dénominateur = self.dénominateur * other.dénominateur
    return Fraction(new_numérateur, new_dénominateur)

def __rmul__(self, other):
    return self.__mul__(other)

def __truediv__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    if other.numérateur == 0:
        raise ZeroDivisionError("Division par zero.")
    return self * Fraction(other.dénominateur, other.numérateur)

def __rtruediv__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return Fraction(other, 1) / self

def __neg__(self):
    return Fraction(-self.numérateur, self.dénominateur)

def __abs__(self):
    return Fraction(abs(self.numérateur), self.dénominateur)

# La fonction 'float()' est utilisée ici uniquement pour l'affichage lisible des
# approximations
# et non pour le calcul interne des fractions, qui reste purement rationnel.
def heron_sqrt(K, initial_guess, num_iterations):
    """
    Génère une suite d'approximations de sqrt(K) en utilisant la méthode de Heron d'
    Alexandrie.
    Tous les calculs sont effectués avec des objets Fraction.

    Arguments:
        K (Fraction): Le nombre dont on veut la racine carrée.
        initial_guess (Fraction): La première approximation (x_0).
        num_iterations (int): Le nombre d'itérations à effectuer.

    Retourne:
        list[Fraction]: Une liste des num_iterations + 1 approximations (incluant x_0
        ).
    """
    if not isinstance(K, Fraction) or not isinstance(initial_guess, Fraction):
        raise TypeError("K et initial_guess doivent être des objets Fraction.")
    if K <= Fraction(0, 1):
        raise ValueError("K doit être strictement positif pour une racine carrée
        réelle.")
    if initial_guess <= Fraction(0, 1):
        raise ValueError("L'approximation initiale doit être strictement positive.")
    if num_iterations < 0:
        raise ValueError("Le nombre d'itérations ne peut pas être négatif.")

    approximations = [initial_guess]
    x_n = initial_guess

    for _ in range(num_iterations):
        # x_{n+1} = (x_n + K/x_n) / 2
        term_K_div_x_n = K / x_n
        sum_terms = x_n + term_K_div_x_n
        x_n_plus_1 = sum_terms / Fraction(2, 1)
        approximations.append(x_n_plus_1)

```

```

        x_n = x_n_plus_1
    return approximations

# Assertions Mathematiques pour heron_sqrt
# Approximation de sqrt(2)
K_val = Fraction(2, 1)
initial = Fraction(1, 1)
iterations = 5
seq_sqrt2 = heron_sqrt(K_val, initial, iterations)

# Verifier la convergence (qualitativement)
# Les termes devraient se rapprocher de sqrt(2) (approx. 1.41421356...)
expected_values = [
    Fraction(1, 1), # 1.0
    Fraction(3, 2), # 1.5
    Fraction(17, 12), # 1.41666...
    Fraction(577, 408), # 1.4142156...
    Fraction(665857, 470832), # 1.4142135623746...
    Fraction(886731088897, 627018906664) # 1.414213562373095...
]

for i in range(len(seq_sqrt2)):
    # print(f"x_{i} = {seq_sqrt2[i]} (approx. {seq_sqrt2[i].numérateur / seq_sqrt2[i].denominateur})")
    assert seq_sqrt2[i] == expected_values[i], f"Assertion echouee a l'iteration {i} pour sqrt(2)."
print("Toutes les assertions pour 'heron_sqrt' (approximation de sqrt(2)) ont reussi.")

# Test pour K = 4 (la racine est un rationnel)
K_val_rational = Fraction(4, 1)
initial_rational = Fraction(1, 1)
iterations_rational = 3
seq_sqrt4 = heron_sqrt(K_val_rational, initial_rational, iterations_rational)
# x0 = 1
# x1 = (1 + 4/1)/2 = 5/2 = 2.5
# x2 = (2.5 + 4/2.5)/2 = (2.5 + 1.6)/2 = 4.1/2 = 2.05
# x3 = (2.05 + 4/2.05)/2 = (2.05 + 1.9512...)/2 = 4.0012.../2 = 2.0006...
assert seq_sqrt4[0] == Fraction(1, 1)
assert seq_sqrt4[1] == Fraction(5, 2)
assert seq_sqrt4[2] == Fraction(41, 20)
print("Toutes les assertions pour 'heron_sqrt' (approximation de sqrt(4)) ont reussi.")

```

Analyse de la Convergence

```

print("\n--- Analyse de la Convergence de la suite pour sqrt(2) ---")
K_val = Fraction(2, 1)
initial = Fraction(1, 1)
iterations = 10
seq_sqrt2_long = heron_sqrt(K_val, initial, iterations)

for i, approx_f in enumerate(seq_sqrt2_long):
    float_approx = approx_f.numerateur / approx_f.denominateur
    # Calcul de l'erreur quadratique: (x_n)^2 - K
    error_sq = (approx_f * approx_f) - K_val
    print(f"x_{i} = {approx_f} (float: {float_approx:.15f})")
    print(f" (x_{i})^2 - 2 = {error_sq} (float: {error_sq.numerateur / error_sq.denominateur:.15f})")

# Verification que la suite est une suite de Cauchy (qualitatif)
# Les termes devraient se rapprocher les uns des autres.
print("\n--- Ecart entre termes consecutifs ---")
for i in range(len(seq_sqrt2_long) - 1):
    diff = abs(seq_sqrt2_long[i+1] - seq_sqrt2_long[i])

```

```

float_diff = diff.numerateur / diff.denominateur
print(f"|x_{i+1} - x_{i}| = {diff} (float: {float_diff:.15f})")
# Assertons que les differences diminuent (ce n'est pas toujours vrai pour les
# premieres iterations,
# mais devient rapidement vrai quand on s'approche de la convergence)
if i > 0: # A partir de la 2eme iteration
    prev_diff = abs(seq_sqrt2_long[i] - seq_sqrt2_long[i-1])
    assert float_diff < prev_diff.numerateur / prev_diff.denominateur, \
        f"La difference ne diminue pas a l'iteration {i} : {float_diff} vs {
            prev_diff.numerateur / prev_diff.denominateur}"
print("Les ecarts entre termes consecutifs diminuent, illustrant la propriete de
    Cauchy de la suite.")

```

Validation Algorithmique Rigoureuse et Assertions Mathématiques : Une suite de Cauchy de rationnels converge vers un irrationnel (dans $\mathbb{R}$)

1. Nature de la suite de Héron d'Alexandrie : La suite (x_n) générée par la méthode de Héron d'Alexandrie pour $K > 0$ est une suite de nombres rationnels si x_0 est rationnel. En effet, si $x_n \in \mathbb{Q}$ et $K \in \mathbb{Q}$, alors $K/x_n \in \mathbb{Q}$, donc $x_n + K/x_n \in \mathbb{Q}$, et enfin $(x_n + K/x_n)/2 \in \mathbb{Q}$. Ainsi, tous les termes x_n de la suite sont des nombres rationnels.

2. Convergence de la suite : Il est bien connu que la suite (x_n) converge vers $\sqrt{K}$. Pour $K = 2$, elle converge vers $\sqrt{2}$. La convergence est quadratique, ce qui signifie que le nombre de chiffres décimaux corrects double approximativement à chaque itération. C'est pourquoi les écarts entre termes consécutifs diminuent très rapidement, comme observé dans les impressions. Cela démontre que la suite (x_n) est une suite de Cauchy : ses termes se resserrent indéfiniment.

3. $\sqrt{2}$ est irrationnel : Comme discuté dans le TP-02, $\sqrt{2}$ n'appartient pas à $\mathbb{Q}$. La démonstration classique par l'absurde suppose $\sqrt{2} = p/q$ sous forme irréductible, puis montre que p et q doivent tous deux être pairs, contredisant l'irréductibilité.

4. Incomplétude de $\mathbb{Q}$: Nous avons donc une suite de nombres rationnels (x_n) qui est une suite de Cauchy mais qui converge vers un nombre irrationnel ($\sqrt{2}$). Ceci est la preuve formelle de l'incomplétude de $\mathbb{Q}$. Si $\mathbb{Q}$ était complet, toute suite de Cauchy de rationnels devrait converger vers un rationnel. Puisqu'elle converge vers un irrationnel, $\mathbb{Q}$ n'est pas complet.

5. Axiome de la Borne Supérieure et $\mathbb{R}$: C'est précisément pour combler cette incomplétude que l'ensemble des nombres réels $\mathbb{R}$ est construit. Dans $\mathbb{R}$, l'axiome de la borne supérieure (ou de manière équivalente, le fait que toute suite de Cauchy de réels converge vers un réel) garantit que de telles suites convergent vers un élément l'ensemble $\mathbb{R}$, qu'il soit rationnel ou irrationnel.

Conclusion du TP-04

Nous avons utilisé la méthode de Héron d'Alexandrie pour générer une suite de nombres rationnels qui converge vers $\sqrt{2}$. L'observation des termes et de leur rapprochement mutuel illustre le concept de suite de Cauchy. Le fait que cette suite de rationnels converge vers un nombre irrationnel ($\sqrt{2}$) est une démonstration concrète et puissante de l'incomplétude de l'ensemble des nombres rationnels. Cela souligne la nécessité et l'importance de l'axiome de la borne supérieure pour définir l'ensemble des nombres réels comme un corps complet où de telles suites trouvent toujours leur limite. Le prochain TP fera le lien définitif entre ces concepts pour démontrer la densité des rationnels dans les réels.

'yaml

TP-05 : L'Axiome de la Borne Supérieure et la Densité de $\mathbb{Q}$ dans $\mathbb{R}$

Introduction

Ce dernier TP réunit les concepts explorés précédemment : la structure de corps ordonné des rationnels, l'incomplétude de $\mathbb{Q}$, la propriété d'Archimède, et l'axiome de la borne supérieure. Notre objectif principal est de démontrer la densité de $\mathbb{Q}$ dans $\mathbb{R}$, une propriété fondamentale qui stipule qu'entre deux nombres réels distincts, il existe toujours un nombre rationnel. Cette propriété est une conséquence directe de l'axiome de la borne supérieure et de la propriété d'Archimède.

Prérequis Mathématiques

Définition 1 (Densité d'un ensemble dans un autre) : Un ensemble A est dit dense dans un ensemble B (où $A \subseteq B$) si pour tout $x, y \in B$ avec $x < y$, il existe un élément $a \in A$ tel que $x < a < y$.

Théorème (Densité de $\mathbb{Q}$ dans $\mathbb{R}$) : Pour tous nombres réels x et y tels que $x < y$, il existe un nombre rationnel q tel que $x < q < y$.

Preuve de la Densité de $\mathbb{Q}$ dans $\mathbb{R}$ (esquisse) : Soient $x, y \in \mathbb{R}$ avec $x < y$. 1. Puisque $y - x > 0$, par la propriété d'Archimède (conséquence 2 du TP-03), il existe un entier naturel n tel que $1/n < y - x$. 2. Considérons le nombre nx . Toujours par la propriété d'Archimède (conséquence 1 du TP-03), il existe un entier m tel que $m > nx$. En fait, on peut trouver un entier m tel que $m - 1 \leq nx < m$. 3. De $m - 1 \leq nx$, on a $(m - 1)/n \leq x$. 4. De $nx < m$, on a $x < m/n$. 5. Nous avons $(m - 1)/n \leq x < m/n$. 6. En utilisant $1/n < y - x$, on a $x + 1/n < y$. 7. Comparons m/n et y : $m/n = (m - 1)/n + 1/n \leq x + 1/n < y$. 8. Donc, nous avons $x < m/n < y$. 9. Puisque m et n sont des entiers et $n \neq 0$, $q = m/n$ est un nombre rationnel. Ceci prouve l'existence d'un rationnel entre x et y .

Exercice 1 : Réitération de l'Axiome de la Borne Supérieure

L'axiome de la borne supérieure n'est pas une fonction implémentable, mais un postulat mathématique. Cependant, nous pouvons le ré-illustrer par un exemple.

Tâche 1.1 : Discussion de l'existence du supremum

Considérez l'ensemble $S = \{x \in \mathbb{Q} \mid x^2 < 3 \text{ et } x > 0\}$. * Dans $\mathbb{Q}$, S est borné supérieurement (par exemple, par 2). Cependant, S n'a pas de supremum dans $\mathbb{Q}$ car $\sqrt{3}$ est irrationnel. garantit* l'existence de $\sup(S)$ et que $\sup(S) = \sqrt{3}$.

```
# Discussion conceptuelle de l'axiome de la borne superieure.
print("--- Reiteration de l'Axiome de la Borne Superieure ---")

# Representons quelques elements de S pour le contexte (utilisant float pour l'
approximation)
s_elements_float = []
# Generer des rationnels p/q tels que (p/q)^2 < 3.0
D_MAX_S = 100 # Denominateur max pour la simulation
for q in range(1, D_MAX_S + 1):
    for p in range(1, 2 * q + 1): # sqrt(3) est entre 1 et 2
        approx_val = p / q
        if approx_val * approx_val < 3.0:
            s_elements_float.append(approx_val)
```

```

if s_elements_float:
    max_s_element = max(s_elements_float)
else:
    max_s_element = None

print(f"Considerons l'ensemble  $S = \{x \in \mathbb{Q} \mid x^2 < 3 \text{ et } x > 0\}$ .".")
print(f"Quelques elements de S (en float, pour illustration) : {s_elements_float[:10]}...")
print(f"Le plus grand element  $x$  trouve dans notre simulation de  $S$  (avec denominateur  $\leq \{D\_MAX\_S\}$ ) est  $\{max\_s\_element\}$ .".")
print(f"Son carre est :  $\{max\_s\_element * max\_s\_element:.10f\}$ ") # devrait etre proche de 3.0, mais < 3.0

print("\n
Validation Algorithmique Rigoureuse et Assertions Mathematiques (Rappel de l'Axiome de la Borne Superieure) :
")
print("1. L'ensemble  $S$  est non-vide (e.g.,  $1 \in S$ ).")
print("2. L'ensemble  $S$  est borne superieurement dans  $\mathbb{Q}$  (e.g., par  $2$ , car  $2^2=4 \not< 3$ , donc tout  $x \in S$  doit etre  $x < 2$ ).")
print("3. La borne superieure de  $S$  est  $\sqrt{3}$ .")
print("4.  $\sqrt{3}$  est un nombre irrationnel, c'est-a-dire  $\sqrt{3} \notin \mathbb{Q}$ .".")
print("Par consequent, dans  $\mathbb{Q}$ ,  $S$  est borne superieurement mais n'admet pas de borne superieure. Ceci est l'incompletude de  $\mathbb{Q}$ .")
print("L'Axiome de la Borne Superieure postule que dans  $\mathbb{R}$ , tout ensemble non-vide et borne superieurement (comme  $S$ ) possede une borne superieure  $\mathbb{R}$ .")
print("C'est cet axiome qui \"remplit les trous\" de la droite numerique, garantissant que  $\sqrt{3}$  existe en tant que nombre reel.")

```

Exercice 2 : Démonstration Pratique de la Densité de $\mathbb{Q}$ dans $\mathbb{R}$

Nous allons implémenter une fonction qui, pour deux réels a et b (représentés par des float), trouve un rationnel q entre eux.

Tâche 2.1 : Fonction trouver_rationnel_entre

Implémentez `trouver_rationnel_entre(a, b)` qui retourne une `Fraction` q telle que $a < q < b$. Cette fonction doit utiliser les principes de la preuve esquissée ci-dessus, s'appuyant sur la propriété d'Archimède.

```

# Re-utiliser la classe Fraction et gcd du TP-01
# Copier-coller le code de Fraction et gcd ici pour l'autonomie du fichier
def gcd(a, b):
    a_abs = abs(a)
    b_abs = abs(b)
    while b_abs:
        a_abs, b_abs = b_abs, a_abs % b_abs
    return a_abs

class Fraction:
    def __init__(self, numerateur, denominateur):
        if denominateur == 0:
            raise ValueError("Le denominateur ne peut pas etre zero.")
        if denominateur < 0:
            numerateur = -numerateur
            denominateur = -denominateur
        common_divisor = gcd(numerateur, denominateur)
        self.numerateur = numerateur // common_divisor
        self.denominateur = denominateur // common_divisor

    def __str__(self):
        if self.denominateur == 1:

```

```

        return str(self.numerateur)
    return f"{self.numerateur}/{self.denominateur}"

def __repr__(self):
    return f"Fraction({self.numerateur}, {self.denominateur})"

def __eq__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return self.numerateur == other.numerateur and self.denominateur == other.denominateur

def __lt__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return self.numerateur
self.denominateur

def __le__(self, other):
    return self < other or self == other

def __gt__(self, other):
    return not (self <= other)

def __ge__(self, other):
    return not (self < other)

def __add__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    new_numerateur = self.numerateur
self.denominateur
    new_denominateur = self.denominateur * other.denominateur
    return Fraction(new_numerateur, new_denominateur)

def __radd__(self, other):
    return self.__add__(other)

def __sub__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    new_numerateur = self.numerateur
self.denominateur
    new_denominateur = self.denominateur * other.denominateur
    return Fraction(new_numerateur, new_denominateur)

def __rsub__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return Fraction(other, 1) - self

def __mul__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    new_numerateur = self.numerateur * other.numerateur
    new_denominateur = self.denominateur * other.denominateur
    return Fraction(new_numerateur, new_denominateur)

def __rmul__(self, other):
    return self.__mul__(other)

```

```

def __truediv__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    if other.numerateur == 0:
        raise ZeroDivisionError("Division par zero.")
    return self * Fraction(other.denominateur, other.numerateur)

def __rtruediv__(self, other):
    if not isinstance(other, Fraction):
        if isinstance(other, int): other = Fraction(other, 1)
        else: return NotImplemented
    return Fraction(other, 1) / self

def __neg__(self):
    return Fraction(-self.numerateur, self.denominateur)

def __abs__(self):
    return Fraction(abs(self.numerateur), self.denominateur)

def trouver_rationnel_entre(a, b):
    """
    Trouve un nombre rationnel q (Fraction) tel que  $a < q < b$ .
    a et b sont des floats representant des nombres reels.
    Cette fonction s'appuie sur la preuve de la densite de Q dans R.
    """
    if not isinstance(a, (float, int)) or not isinstance(b, (float, int)):
        raise TypeError("a et b doivent etre des nombres reels (float ou int).")
    if a >= b:
        raise ValueError("a doit etre strictement inferieur a b.")

    # Etape 1: Trouver n tel que  $1/n < b - a$  (Propriete d'Archimede)
    #  $n * (b - a) > 1$ 
    diff = b - a

    # Implementation de verifier_archimede (du TP-03) pour ce besoin.
    def verifier_archimede_local(x, y):
        n = 0
        while n * x <= y:
            n += 1
        return n

    n_val = verifier_archimede_local(diff, 1.0) # n tel que  $n * (b - a) > 1$ 

    # Etape 2: Trouver m tel que  $m-1 <= n$ 
    # m doit etre l'entier suivant immediatement n*a
    # Python's int() tronque, nous avons besoin de floor.
    # Pour un x, floor(x) est l'entier n tel que  $n <= x < n+1$ .
    # Nous voulons  $m-1 <= n$ 
    # Donc  $m = \text{floor}(n*a) + 1$ .

    # Implementation de partie_entiere (du TP-03) pour ce besoin.
    def partie_entiere_local(x):
        if x >= 0:
            return int(x)
        else:
            if x == int(x):
                return int(x)
            else:
                return int(x) - 1 # floor pour les negatifs non entiers

    m_minus_1 = partie_entiere_local(n_val * a)
    m_val = m_minus_1 + 1

```

```

# Etape 3: Le rationnel q = m/n
q = Fraction(m_val, n_val)

return q

# Assertions Mathematiques pour trouver_rationnel_entre
print("\n--- Tests de 'trouver_rationnel_entre' (Densite de Q dans R) ---")

# Cas 1: entre deux entiers
a1, b1 = 2.0, 3.0
q1 = trouver_rationnel_entre(a1, b1)
print(f"Entre {a1} et {b1}, rationnel trouve: {q1} (float: {q1.numerateur / q1.denominateur})")
assert a1 < (q1.numerateur / q1.denominateur) < b1, f"Assertion echouee: {a1} < {q1} < {b1}"

# Cas 2: entre un entier et un irrationnel
a2, b2 = 1.0, 1.41421356237 # sqrt(2)
q2 = trouver_rationnel_entre(a2, b2)
print(f"Entre {a2} et {b2}, rationnel trouve: {q2} (float: {q2.numerateur / q2.denominateur})")
assert a2 < (q2.numerateur / q2.denominateur) < b2, f"Assertion echouee: {a2} < {q2} < {b2}"

# Cas 3: entre deux irrationnels (approximes par float)
a3, b3 = 1.41421356237, 1.73205081 # sqrt(2) et sqrt(3)
q3 = trouver_rationnel_entre(a3, b3)
print(f"Entre {a3} et {b3}, rationnel trouve: {q3} (float: {q3.numerateur / q3.denominateur})")
assert a3 < (q3.numerateur / q3.denominateur) < b3, f"Assertion echouee: {a3} < {q3} < {b3}"

# Cas 4: intervalle tres petit
a4, b4 = 0.3333333333333333, 0.3333333333333334 # 1/3 et un peu plus
q4 = trouver_rationnel_entre(a4, b4)
print(f"Entre {a4} et {b4}, rationnel trouve: {q4} (float: {q4.numerateur / q4.denominateur})")
assert a4 < (q4.numerateur / q4.denominateur) < b4, f"Assertion echouee: {a4} < {q4} < {b4}"

# Cas 5: nombres negatifs
a5, b5 = -2.5, -2.4
q5 = trouver_rationnel_entre(a5, b5)
print(f"Entre {a5} et {b5}, rationnel trouve: {q5} (float: {q5.numerateur / q5.denominateur})")
assert a5 < (q5.numerateur / q5.denominateur) < b5, f"Assertion echouee: {a5} < {q5} < {b5}"

# Cas d'erreur: a >= b
try:
    trouver_rationnel_entre(1.0, 1.0)
    assert False, "Assertion echouee: ValueError attendue pour a >= b"
except ValueError:
    assert True, "ValueError correctement levee pour a >= b"

print("Toutes les assertions pour 'trouver_rationnel_entre' ont reussi.")

```

Validation Algorithmique Rigoureuse et Assertions Mathématiques : La preuve de la densité de $\mathbb{Q}$ dans $\mathbb{R}$ est formellement vérifiée.

Notre fonction `trouver_rationnel_entre(a, b)` implémente fidèlement les étapes de la preuve formelle de la densité de $\mathbb{Q}$ dans $\mathbb{R}$.

1. Utilisation de la Propriété d'Archimède (TP-03) : L'étape cruciale est de trouver un entier n tel que $1/n < b - a$. Cela est rendu possible par la propriété d'Archimède, qui assure que pour tout $\epsilon > 0$ (ici $\epsilon = b - a$),

il existe un n tel que $1/n < \epsilon$. Notre fonction `verifier_archimede_local(diff, 1.0)` calcule précisément ce n . Si n n'existait pas, cela signifierait que $b - a$ est "infinitement petit", ce qui est contredit par la propriété d'Archimède.

2. Construction du Rationnel : Une fois n trouvé, nous cherchons un entier m tel que $m - 1 \leq na < m$. Cette étape utilise l'existence de la fonction "partie entière" pour tout nombre réel (ici na), qui est également une conséquence de la propriété d'Archimède. Notre fonction `partie_entiere_local(n_val * a)` détermine $m - 1$, et donc m . Le nombre $q = m/n$ est alors construit. Par définition, m et n sont des entiers et $n \neq 0$, donc q est bien un nombre rationnel.

3. Vérification de l'encadrement : La preuve formelle montre que $x < m/n < y$. Notre code ne fait que construire q et les assertions valident que le q trouvé respecte bien $a < q < b$. L'utilisation de `float` pour a et b est une approximation pratique des nombres réels, et les calculs peuvent subir des erreurs de précision, mais le principe mathématique sous-jacent est solidement établi. * On a $m - 1 \leq na$. Donc $(m - 1)/n \leq a$. * Par définition de m , on a $na < m$, donc $a < m/n$. C'est le premier côté de l'inégalité $a < q$. * Il reste à montrer que $m/n < b$. On sait que $(m - 1)/n \leq a$. * On a aussi $1/n < b - a$. * Additionnons les inégalités $(m - 1)/n \leq a$ et $1/n < b - a$: $(m - 1)/n + 1/n < a + (b - a)$ $m/n < b$. C'est le second côté de l'inégalité $q < b$.

En combinant $a < m/n$ et $m/n < b$, nous obtenons $a < q < b$. La démonstration est complète et notre code en est une illustration fidèle.

Conclusion du TP-05

Ce dernier TP a conclu notre exploration du "Jalon 13" en démontrant la densité de $\mathbb{Q}$ dans $\mathbb{R}$. Nous avons vu comment cette propriété fondamentale découle directement de l'axiome de la borne supérieure (qui assure que $\mathbb{R}$ ne contient pas de "trous" comme $\mathbb{Q}$) et de la propriété d'Archimède (qui empêche l'existence de nombres infiniment grands ou infiniment petits, permettant de "zoom-in" sur n'importe quel intervalle). L'implémentation de la fonction `trouver_rationnel_entre` a servi de preuve constructive de cette densité.

Ensemble, ces cinq TPs ont permis de reconstruire, de manière pratique et rigoureuse, les fondations axiomatiques qui définissent la structure des nombres réels, essentielles à toute l'analyse mathématique.